PROJECT PROPOSAL

SHOP INVENTORY AND BILLING SYSTEM

Object-Oriented Programming (C++)

STUDENT	ROLL NO	SUBMITTED TO
Mahveen Sohail	**25230**	**Atif Masih**

SELECTED PROJECT

> **Shop Inventory and Billing System** – a console-based C++ project of my own choice (not from the given list of 40). It combines ideas from three listed projects: Product Inventory, Shopping Cart, and Payroll/Billing.

PROBLEM STATEMENT

Many small shops in Pakistan, such as general stores, pharmacies, and stationery shops, still keep their stock and sales records in a notebook or on paper. This causes many problems:

- Stock counting mistakes: the shopkeeper does not know exactly how many items are left.
- Items finish without warning, and the shop loses sales.
- Making bills by hand is slow, and calculation mistakes are common.
- Daily sales records are hard to find and total at the end of the day.
- Paper records can be lost or damaged.

Shop owners need a simple, fast, and low-cost tool to manage their products and make bills correctly.

PROPOSED SOLUTION

I will build a C++ program that manages products, stock, customers, and bills in one place. The shopkeeper can add products, search them, make a bill for a customer, and see daily sales. All data will be saved in files, so nothing is lost when the program closes. I will use Object-Oriented Programming so that the code is clean, organized, and easy to extend later (for example, adding a GUI or a web version).

REQUIRED FEATURES AND TASKS

1. **Product management:** add, update, delete, view, and search products (by name or ID).
2. **Stock management:** update stock after every sale, and show a low-stock alert when quantity is below a limit.
3. **Billing:** create a bill with many items and quantities, apply discount, calculate total, and print the bill.
4. **Customer record (basic):** save customer name and bill history.

5. **Sales report:** show total sales of the day and list of bills.
6. **File handling:** save and load products and bills from files.
7. **Input validation:** stop wrong inputs (negative price, wrong ID, quantity more than stock).
8. **Simple login (optional):** admin/cashier access if time allows.

HOW I WILL USE OOP CONCEPTS

> **Tools and Technologies:** C++ language; VS Code or Code::Blocks as the editor/IDE with the g++ compiler; Git and GitHub for version control and documentation.

OOP concepts and how I will use them:

Concept	How I will use it
Classes and Objects	Product, Customer, Bill, and Inventory will be separate classes.
Encapsulation	Data like price and stock will be private, and changed only through methods with checks.
Inheritance	Different product types (for example, GroceryItem, MedicalItem) will inherit from a base Product class.
Polymorphism	Different discount or tax rules will use virtual functions, so each type calculates its own price.
Abstraction	An abstract class will define what every product must provide (price, details), hiding the extra work.
Constructors and Destructors	Used to set up objects correctly and clean memory.
Operator overloading (optional)	For example, comparing or printing products and bills easily.
File handling	Saving and loading object data from files.
Vectors / STL	Storing lists of products and bill items.

LOGIC BUILDING FOCUS

- Calculating bill totals with quantity, discount, and tax.
- Checking stock before a sale and updating it after the sale.
- Searching and sorting products (by name, price, or quantity).
- Handling wrong or unexpected input without crashing the program.
- Reading and writing records in files in the correct format.
- Breaking a big problem into small functions and classes.

SUGGESTED CLASSES

- **Product** (base class): id, name, price, quantity, category.
- **GroceryItem / MedicalItem** (derived classes): extra details like expiry date or tax rule.
- **Inventory:** keeps all products; add, remove, search, and low-stock check.

- **Customer:** name, phone, and purchase history.
- **BillItem:** one product, its quantity, and line total.
- **Bill:** bill number, date, list of BillItems, discount, and final total.
- **FileManager:** saves and loads data from files.
- **Menu / ShopSystem:** the main class that controls the program menu.

DIFFICULTY

> **Medium to Hard.** It needs many connected classes, file handling, and careful logic, but it is still possible to complete in the available time if I build one feature at a time.

EXPECTED OUTCOME AND BENEFITS

- A working C++ program that can manage products and stock and make correct bills.
- Data saved in files and loaded again when the program starts.
- A GitHub repository with clean code, regular commits, and a clear README (with project description, features, screenshots, and how to run it).
- Good practice of all main OOP concepts in one real project.

Benefits:

FOR ME I will learn how to design a real project with OOP, not only small examples.	**FOR CAREER** The GitHub project will be part of my portfolio for internships and freelancing.
FOR REAL USE Small shops can use the program to save time and avoid mistakes.	**FOR FUTURE** It can be improved into a GUI or web application, and may become a useful product.